

CACL: Context-Aware Contrastive Learning for Cross-Entity Semantic Modeling in Anomaly Detection and Explanation

September 2, 2025

1 Introduction

Structured data analysis is often more feasible and computationally efficient than unstructured alternatives. As a result, real-world datasets are commonly represented as, or transformed into, vectors (records) of feature–value pairs organized in tabular formats with rows and columns. Ensuring the integrity of such data is critical for downstream tasks such as diagnosis, decision-making, forecasting, and scientific discovery. *Anomaly detection* aims to identify records that deviate significantly from the patterns exhibited by normal data. These anomalies often correspond to critical issues including medical errors [1], fraudulent behavior [2], equipment faults [3], or rare yet important events [4, 5, 6]. We distinguish four kinds of deviations: **Type 1 anomalies** — violations of *feature-level constraints* within a single feature (e.g., `age < 0` or `blood_pressure` outside the expected range). Such errors are relatively easy to detect with rule-based or thresholding techniques. **Type 2 anomalies** — *logically inconsistent combinations* of otherwise valid feature values (e.g., `driver_license_issued = true` with `age = 12`). Because each attribute appears valid in isolation, these anomalies are far more subtle, especially in high-dimensional or unlabeled datasets. **Type 3 anomalies** — violations of *record-level constraints across rows*. They occur in time-series, sequential, or spatially correlated data where each row is locally valid yet becomes invalid when viewed in context. For instance, an infant’s *patient_weight* should increase by 4–22 lb between successive check-ups; a single measurement may pass all instantaneous checks but, together with earlier records, imply an impossible growth trajectory. Detecting such anomalies requires context-aware models. **Type 4 noise** — benign, stochastic perturbations that slightly distort otherwise valid records without violating any semantic rule. Examples include sensor jitter, rounding error, transient placeholders (“–1”, “N/A”), or minor typos (“diabetes” vs. “diabetis”). Because these deviations are harmless, a robust detector should treat them as normal, not anomalous.

Challenges. These observations give rise to two core challenges in modern anomaly detection:

1. Comprehensive and generalizable detection of anomalies: Existing models struggle to detect complex semantic inconsistencies (Type 2 and Type 3 anomalies), particularly when such patterns have not been explicitly observed during training. This challenge is amplified in real-world applications where labeled anomalies are scarce or entirely unavailable. In fact, even human experts may miss such anomalies without a clear understanding of the underlying feature dependencies.

2. Lack of interpretability: Even when anomalies are flagged, they are typically reported as opaque scores with no insight into which rules or feature combinations were violated. This limits trust, debuggability, and the ability to take informed corrective action.

Objectives. This paper proposes a unified, model-agnostic framework for comprehensive, generalizable and interpretable anomaly detection. The approach is organized into two coordinated objectives:

Objective 1: Comprehensive and Generalizable Anomaly Detection. We develop a self-supervised, model-agnostic framework that learns rich intra- and inter-record dependencies, enabling the detection of feature-level, cross-feature, and record-level anomalies in unlabeled data even when the underlying violations were not encountered during training. The proposed solution integrates a multi-view and context-aware contrastive objective jointly supporting comprehensive, generalizable, and label-free detection of Type 1–3 anomalies.

Objective 2: Explainability and Interpretability. To make anomaly detection actionable, we propose a self-supervised approach that explains each anomaly by identifying violated relationships between feature and record groups. Unlike existing anomaly explanation methods that rely on post-hoc analysis, where explanations are generated after the

detector is trained and may not align with the model’s decision boundaries, the proposed method is built directly on the model developed in objective 1. It identifies conflicting feature partitions and extracts internal dependency patterns from inliers. All explanations are derived from the same model used for detection, ensuring coherence, interpretability, and practical value.

2 Objective 1: Anomaly Detection

The goal of this project is to develop a *model-agnostic, self-supervised* learning anomaly detection framework that can accurately identify *all forms of anomalies* including attribute-level (Type 1), cross-feature (Type 2), and record-level (Type 3) *without* relying on labeled anomalies or hand-crafted rules. We aim to capture high-order dependencies across large sets of features and records.

Related Work. Research on data irregularities spans three technical paradigms, each aligned with a different anomaly type. **Type-1 anomalies (feature-level violations):** Classic statistical and distance-based detectors—e.g., z-score and Grubbs’ tests [7], Tukey fences, one-class SVM [8], Isolation Forest [9], and subspace PCA [10]—work well when a single attribute is obviously flawed. However, they ignore cross-attribute interactions, so they miss composite errors and offer little leverage when attribute distributions evolve. **Type-2 anomalies (cross-feature inconsistencies):** Deep autoencoders [11], variational autoencoders [12], normalizing flows [13], and diffusion models [14] score a record via reconstruction error or (negative) log-likelihood, implicitly modelling dependencies among attributes. Recent masking-based transformers refine this idea. *Masked Cell Modeling (MCM)* [15] hides random feature subsets and reconstructs them, encouraging pair-wise correlation capture but providing no row-level separation in latent space. *Mask-Aware Prototype Learning (PTAD)* [16] adds orthogonal prototypes and an optimal-transport loss, yet still relies on exactly two masked views, limiting its ability to model higher-order feature groups. *Partition Contrastive Learning (PCLAD)* [17] was introduced to overcome these reconstruction-centric limitations by explicitly modelling inter-feature consistency with a contrastive objective: it splits each record into two complementary partitions and applies an InfoNCE loss that attracts embeddings from the same record while repelling those from different records. This contrastive design removes reliance on reconstruction and begins to capture richer intra-record structure than masking approaches. Nevertheless, its strict two-view setup cannot express higher-order feature interactions and treats every other record as a negative, even when genuine semantic similarity exists, so it still struggles to detect *Type-3 anomalies*. **Type-3 anomalies (record-level violations):** Sequence models such as LSTM autoencoders, USAD [18], InterFusion [19], and prediction-error baselines (ARIMA, TCNs) detect sequential irregularities but usually treat each feature independently in the latent space. The Anomaly Transformer [20] compares attention distributions between two consecutive windows, giving limited pairwise context, while ignoring rich cross-feature structure within each window.

While the proposed framework builds on recent contrastive-based approaches to model rich cross-feature dependencies, those techniques still exhibit shortcomings that this study is designed to overcome:

1. **Binary-partition bottleneck.** Existing contrastive methods create exactly two views per record, limiting their ability to capture higher-order dependencies in wide, heterogeneous tables.
2. **Blindness to record-level (Type 3) anomalies.** By treating every cross-record pair as a negative example, these methods ignore legitimate temporal, spatial, or cohort relationships; consequently, anomalies that violate such context-dependent patterns remain undetected.

2.1 Proposed Method

Building on the strengths of contrastive and cross-feature representation learning, we introduce a unified, self-supervised framework that captures complex dependencies both within and across records. The solution integrates multi-view semantic partitioning and context-aware contrastive learning enabling rich cross-feature and inter-record modeling of semantic relationships.

2.1.1 Addressing Limitation 1: Multi-View Semantic Partition Representation Learning

Step 1: Generate K feature views. For every unlabeled record $x \in \mathbb{R}^D$, the framework samples $K > 2$ *disjoint* feature subsets: $x = [x^{(1)}, x^{(2)}, \dots, x^{(K)}]$, where each $x^{(k)}$ is a semantically grouped slice of the original vector. To preserve meaningful interactions, partitions are derived once at the start of training from a data-driven feature-affinity matrix and are kept fixed thereafter. The framework first computes a symmetric matrix $G \in \mathbb{R}^{D \times D}$, where $G_{ij} = \frac{1}{2}(A_{ij} + A_{ji})$. A scalable graph partitioner—such as spectral clustering with Nyström approximation or community detection methods like Leiden—is then applied to yield K coherent groups, $\{x^{(1)}, \dots, x^{(K)}\}$.

Step 2: Encode each view with a shared backbone. A single encoder f_θ processes each view, $h^{(k)} = g_\phi(f_\theta(x^{(k)}))$, $k = 1, \dots, K$, followed by a projection head g_ϕ . Embeddings are *L2-normalized* to ensure stable cosine similarity comparisons and to prevent the energy head from exploiting vector magnitudes.

2.1.2 Addressing Limitation 2: Context-Aware InfoNCE for Record-Level Dependencies

Binary row-independence assumptions break down when real dependencies (temporal, spatial, cohort) exist. The PI therefore augments the contrastive scheme with a **context-aware InfoNCE** that treats rows as positives or negatives *relative to* a learned or domain-informed context.

Step 1: Build context groups. Context groups $\mathcal{G}$ are defined based on the data modality. For static data, they correspond to clusters in the frozen embedding space. For sequential data, they are formed using sliding windows of size w . For spatial data, context groups are defined as fixed-radius or k -nearest-neighbor (k-NN) neighborhoods. These groups may be periodically updated after encoder fine-tuning.

Step 2: Context-aware Contrastive Loss. To detect *Type 3 anomalies*, the PI extends the local multi-view InfoNCE loss ($\mathcal{L}_{mv}$) to incorporate *context-based* positives. For every anchor view $h_i^{(k)}$ (partition k of record i), the PI defines

$$\mathcal{P}_{i,k} = \underbrace{\mathcal{P}_{i,k}^{mv}}_{\text{other partitions same record}} \cup \underbrace{\mathcal{P}_{i,k}^{ctx}}_{\text{partitions context rows}}, \quad \mathcal{N}_{i,k} = \text{all other views in the minibatch},$$

where $\mathcal{P}_{i,k}^{mv} = \{h_i^{(l)}\}_{l \neq k}$ and $\mathcal{P}_{i,k}^{ctx} = \{h_j^{(l)} \mid j \in \mathcal{G}(i)\}$, with $\mathcal{G}(i)$ denoting the context group (e.g., a sliding time window, spatial neighborhood, or learned cluster) of record i . The resulting context-aware contrastive loss is:

$$\mathcal{L}_{ctx+mv} = -\frac{1}{|\mathcal{B}|} \sum_{(i,k) \in \mathcal{B}} \log \frac{\sum_{(j,l) \in \mathcal{P}_{i,k}} w_{(j,l)} \exp(\text{sim}(h_i^{(k)}, h_j^{(l)})/\tau)}{\sum_{(j,l) \in \mathcal{N}_{i,k}} \exp(\text{sim}(h_i^{(k)}, h_j^{(l)})/\tau)},$$

with weights $w_{(j,l)} = \alpha_{mv}$ for intra-record positives and $w_{(j,l)} = \alpha_{ctx}$ for context positives ($\alpha_{mv} \geq \alpha_{ctx}$). By simultaneously aligning views from the **same record** and from **contextually related records**, the encoder learns a representation space in which cross-record regularities are preserved. As a result, records that abruptly diverge from their context, such as unrealistic temporal jumps or out-of-distribution neighbors, experience weakened alignment and are more likely to be flagged as Type 3 anomalies due to elevated anomaly scores.

Stochastic Context-Aware Contrastive Loss. The original context-aware contrastive loss function computes pairwise cosine similarities between each anchor embedding $h_i^{(k)}$ and all partitioned views $h_j^{(l)}$ from both intra-record positives and all members of the context group $\mathcal{G}(i)$. While theoretically rich in contrastive signal, this approach has time complexity $\mathcal{O}(B^2K)$ per batch due to dense comparisons across all batch elements and their multiple views. When context groups are large or batch sizes increase, this cost becomes prohibitive.

To alleviate this, we introduce a stochastic sampling strategy for context positives. Instead of using all members in the context group $\mathcal{G}(i)$ during training, we randomly sample a fixed-size subset of M members from the available group within the current batch:

$$\mathcal{P}_{i,k}^{ctx} = \text{Sample}(\{h_j^{(k)} : j \in \mathcal{G}(i), j \neq i\} \cap \text{batch}, M)$$

This reduces the number of context comparisons from $\mathcal{O}(|\mathcal{G}(i)|K)$ to $\mathcal{O}(MK)$ per anchor view, significantly decreasing compute and memory requirements without eliminating the benefits of relational modeling. In our implementation, we set $M = 3$ as a default.

This modification preserves the core semantics of the context-aware loss by maintaining view alignment within semantically similar samples, while enabling efficient scaling to larger batches and deeper models.

3 Objective 2: Explainability and Interpretability

While identifying anomalous samples is critical for ensuring data quality and reliability, it is equally important to understand *why* a particular record is flagged as anomalous. This need for explanation is especially acute for Type 2

and Type 3 anomalies, which involve complex cross-feature or record-level violations that may not be evident from the individual attribute values. Objective 2 addresses this gap by enhancing the explainability and interpretability of anomaly decisions, with the goal of making outputs both actionable and trustworthy. Given a detected anomaly x with a high anomaly score $\mathcal{E}(x)$, the objective is to generate interpretable justifications that reveal: (1) which feature relationships or statistical rules have been violated and (2) which partitions or feature groups exhibit the strongest disagreement.

Related Work. Research on explaining anomaly-detection decisions has converged around three main groups of techniques. **Feature-attribution explanations:** Generic post-hoc explainers such as LIME [21] and SHAP [22] approximate a black-box detector locally with a surrogate model and attribute the anomaly score to individual features. Extensions like DeepSHAP [23] and GradientSHAP [24] adapt these ideas to deep detectors, while other approaches [25] target unsupervised autoencoders. Although widely used, attribution scores tell *what* features were important but not *which rule* or relationship was violated. **Rule-based and neuro-symbolic explanations:** Association-rule mining has been applied to flag violated constraints after detection (e.g., ERminer [26] in electronic health records). LogicAL [27] synthesizes logical counterexamples for image anomalies, while the model by Zhang et al. [28] and the PI’s prior work (ADQuaTe [29] and IDEAL [30]) distill decision trees from autoencoders to expose implicit rules. More recently, LLM-driven approaches such as LogiCode [31] prompt GPT-4 with candidate rules to generate human-readable rationales. These methods provide rule-level insight, but most require hand-crafted templates or domain labels, limiting scalability. **Prototype, counterfactual, and concept-based explanations.** ProtoDash [32] and ACE [33] identify prototypical normal samples and show how a query deviates. Counterfactual explainers (DiCE [34], CounterGAN [35]) search for minimal edits that flip the detector’s decision. Concept attribution frameworks such as TCAV [36] map latent dimensions to user-defined semantic concepts. While intuitive, prototype and counterfactual methods often ignore inter-feature dependencies, and concept approaches demand labeled concept sets.

Objective 2 is motivated by the following key limitations:

1. **Post-hoc detachment.** Most explainers operate after the detector is trained, offering no guarantee that the explanation aligns with the model’s decision.
2. **Rule granularity gap.** Existing feature-attribution tools highlight individual attributes, whereas end-users need violated *rules or relationships*.
3. **Lack of unsupervised compatibility.** Many state-of-the-art techniques assume labeled data, predefined concepts, or supervised losses.

3.1 Proposed Method

To address the limitations of post-hoc, low-granularity, and label-dependent explanation techniques, we introduce a unified, self-supervised pipeline that derives structured, human-interpretable justifications directly from the same representations used for anomaly detection in objective 1. Rather than relying on external rule templates or supervised concepts, this method identifies which feature partitions most strongly disagree and extracts latent dependency patterns from inlier behavior. Each explanatory element including conflicting views and violated relationships generated using the model’s own learned structure, enabling end-to-end consistency and interpretability without labeled data.

Step 1: Pinpoint the most conflicting partitions. The objective is to identify the most prominent source of inconsistency within a flagged record by measuring internal divergence between its semantic partitions and between the record and its context group. For a flagged record x , the framework maintains K semantic partition embeddings $\{h^{(k)}\}_{k=1}^K$, each derived from a distinct subset of features. A pairwise disagreement matrix is computed as: $D_{kl} = 1 - \text{sim}(h^{(k)}, h^{(l)})$, where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity. The most divergent pair of feature partitions is identified by: $(k^*, l^*) = \arg \max_{k < l} D_{kl}$, highlighting the two semantic groups within the record that disagree most. This provides an instance-specific pointer to where the anomaly manifests, using the same latent representations employed during detection.

For record-level anomalies, the framework also considers the context group $\mathcal{G}(x)$ associated with the flagged record. Each member of the group is compared to the flagged record via its average embedding: $h^{(\text{avg})}(x) = \frac{1}{K} \sum_{k=1}^K h^{(k)}$, $d_{x,x'} = 1 - \text{sim}(h^{(\text{avg})}(x), h^{(\text{avg})}(x'))$, $x' \in \mathcal{G}(x)$. The maximum $d_{x,x'}$ or the average context deviation indicates whether the record is misaligned with its peers. This step thus localizes the most salient intra-record or context-level conflict in the flagged sample.

Step 2: Extract self-supervised dependency statements. The objective is to identify which statistical relationships learned from the training data are violated by a flagged record and to express those violations as plain-language dependency statements. This step operates under the assumption that the majority of the training data is normal, with anomalies comprising a minority. To identify violated cross-feature dependencies, we use the same pairwise disagreement scores computed in training. For each inlier record x_i , a binary matrix $\Lambda^{(i)} \in \{0, 1\}^{K \times K}$ is constructed based on a consistency threshold η : $\Lambda_{kl}^{(i)} = \mathbb{1}\{D_{kl}^{(i)} < \eta\}$, $D_{kl}^{(i)} = 1 - \text{sim}(h_i^{(k)}, h_i^{(l)})$. A probabilistic dependency matrix is obtained by aggregating across N training records: $P_{kl} = \frac{1}{N} \sum_{i=1}^N \Lambda_{kl}^{(i)}$, so that each entry P_{kl} represents how frequently partitions k and l agree across the training distribution.

For a flagged record x , any pair (k, l) such that $P_{kl} > \rho$ and $D_{kl}(x) > \eta$ is interpreted as a violated dependency. This yields interpretable output, such as: “*Violated: Partition k is normally consistent with Partition l .*”

In parallel, to explain record-level inconsistencies, the framework tracks the empirical distribution of context alignment scores: $a_i = \frac{1}{|\mathcal{G}(x_i)|} \sum_{x_j \in \mathcal{G}(x_i)} \text{sim}(h^{(\text{avg})}(x_i), h^{(\text{avg})}(x_j))$, where x_i ranges over inlier records. If a flagged record x exhibits a context similarity score a_x significantly below the training distribution (e.g., below a predefined quantile), the system reports that the record violates typical group-level consistency: “*This record is inconsistent with others in its context group.*”

4 Implementation Pipeline for Context-Aware Contrastive Anomaly Detection

To evaluate the effectiveness of the proposed self-supervised anomaly detection framework under various settings, we implement a modular training pipeline that supports both standard and context-aware contrastive learning. This section details each component of the training algorithm, which is responsible for preparing feature partitions, modeling context, training a multi-view encoder, and computing the loss functions.

4.1 Step 1: Semantic Partitioning via Feature Correlation

Given a training dataset $X \in \mathbb{R}^{N \times D}$ with N records and D features, the first step is to divide the features into K disjoint, semantically coherent partitions. This is essential to expose cross-feature dependencies during training. We follow a data-driven procedure based on unsupervised structure in the correlation matrix:

1. Compute the Pearson correlation matrix $\text{Corr}(X^\top) \in \mathbb{R}^{D \times D}$ between features.
2. Apply PCA to reduce the correlation structure into K dimensions:

$$G = \text{PCA}_K(\text{Corr}(X^\top)) \in \mathbb{R}^{D \times K}$$

3. Cluster the rows of G using K -means, grouping features into K partitions $\{\mathcal{P}_1, \dots, \mathcal{P}_K\}$.

Illustrative Example. Suppose we have a dataset with $D = 6$ features:

$$\text{Corr}(X^\top) = \begin{bmatrix} 1 & 0.9 & 0.8 & 0.1 & 0.2 & 0.0 \\ 0.9 & 1 & 0.85 & 0.15 & 0.25 & 0.1 \\ 0.8 & 0.85 & 1 & 0.05 & 0.1 & 0.2 \\ 0.1 & 0.15 & 0.05 & 1 & 0.8 & 0.9 \\ 0.2 & 0.25 & 0.1 & 0.8 & 1 & 0.85 \\ 0.0 & 0.1 & 0.2 & 0.9 & 0.85 & 1 \end{bmatrix}$$

PCA projection with $K = 2$ may yield:

$$G = \begin{bmatrix} 1.5 & 0.2 \\ 1.6 & 0.1 \\ 1.4 & 0.3 \\ -1.3 & 0.5 \\ -1.4 & 0.4 \\ -1.5 & 0.6 \end{bmatrix}$$

KMeans clustering would then separate features $\{f_1, f_2, f_3\}$ and $\{f_4, f_5, f_6\}$ into two partitions. This captures the block

structure of dependencies visible in the correlation matrix.

4.2 Step 2: Building Context Groups

To model inter-record similarity, we compute k -nearest neighbors in the input space using Euclidean distance. For each record x_i , we define a context group:

$$\mathcal{G}(i) = \text{kNN}(x_i; X_{\text{train}}, k)$$

This group is used during training as a source of semantically similar samples for contrastive alignment. The number of neighbors k is typically small (e.g., $k = 5$).

4.3 Step 3: Multi-View Encoder Architecture

Each partition $\mathcal{P}_k$ is processed by a dedicated encoder $f^{(k)}$ and projection head $g^{(k)}$. We implement each encoder as a shallow multilayer perceptron (MLP):

$$\begin{aligned} f^{(k)}(x^{(k)}) &= \text{ReLU}(W_2^{(k)}(\text{ReLU}(W_1^{(k)}x^{(k)} + b_1^{(k)})) + b_2^{(k)}) \in \mathbb{R}^d \\ g^{(k)}(z^{(k)}) &= \frac{W^{(k)}z^{(k)} + b^{(k)}}{\|W^{(k)}z^{(k)} + b^{(k)}\|_2} \in \mathbb{R}^d \end{aligned}$$

Here, $x^{(k)}$ denotes the feature subvector for partition k , $z^{(k)} = f^{(k)}(x^{(k)})$, and $h^{(k)} = g^{(k)}(f^{(k)}(x^{(k)}))$ is the final L2-normalized embedding. All encoders and projectors share the same output dimension d . These embeddings are used during training to maximize similarity between consistent views and minimize similarity to unrelated ones.

4.4 Step 4: Contrastive Training Objective

We train using a contrastive loss based on InfoNCE. Given a batch of B samples, each with K partitions, we compute embeddings $\{h_i^{(k)}\}$ and define positives and negatives.

Standard Multi-View Loss. The default loss aligns views of the same record while repelling all others:

$$\mathcal{L}_{\text{mv}} = -\frac{1}{BK} \sum_{i=1}^B \sum_{k=1}^K \log \frac{\sum_{l \neq k} \exp\left(\frac{\text{sim}(h_i^{(k)}, h_i^{(l)})}{\tau}\right)}{\sum_{j \neq i} \sum_{l=1}^K \exp\left(\frac{\text{sim}(h_i^{(k)}, h_j^{(l)})}{\tau}\right)}$$

Context-Aware Extension. If context is enabled, we include context neighbors as soft positives:

$$\begin{aligned} \mathcal{P}_{i,k} &= \underbrace{\{h_i^{(l)}\}_{l \neq k}}_{\text{intra-record}} \cup \underbrace{\{h_j^{(k)} \mid j \in \mathcal{G}(i)\}}_{\text{context}} \\ \mathcal{L}_{\text{ctx+mv}} &= -\frac{1}{BK} \sum_{i,k} \log \frac{\sum_{(j,l) \in \mathcal{P}_{i,k}} w_{(j,l)} \exp\left(\frac{\text{sim}(h_i^{(k)}, h_j^{(l)})}{\tau}\right)}{\sum_{(j,l) \in \mathcal{N}_{i,k}} \exp\left(\frac{\text{sim}(h_i^{(k)}, h_j^{(l)})}{\tau}\right)} \end{aligned}$$

with weights $w_{(j,l)} = \alpha_{\text{mv}}$ for intra-record and α_{ctx} for context positives.

4.5 Step 5: Optimization and Training Loop

We train for multiple epochs using the Adam optimizer with learning rate $\eta = 10^{-3}$. In each mini-batch, we compute the loss, perform backpropagation, and update parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

The training loss is recorded per epoch to monitor convergence. This modular training routine allows flexible ablation over loss types (context-aware vs. standard) and view configurations (e.g., $K = 2$ vs. $K = 4$).

4.6 Step 6: Anomaly Detection via Embedding Deviations

After training the multi-view model using contrastive learning, we derive anomaly scores for test records based on how their latent representations deviate from the expected distribution. The model does not include an explicit output layer for anomaly classification; instead, it produces K normalized embeddings $\{h^{(k)}\}_{k=1}^K$ for each test record, which are used to compute semantic consistency.

Average Embedding. We first compute the mean embedding across all partitions for each test record:

$$h^{\text{avg}}(x) = \frac{1}{K} \sum_{k=1}^K h^{(k)}(x)$$

Global Mean Embedding. Next, we compute the global centroid across the entire test set:

$$\mu = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} h^{\text{avg}}(x_i)$$

Anomaly Score. The anomaly score is the cosine distance between each record’s average embedding and the global mean:

$$\mathcal{A}(x_i) = 1 - \text{sim}(h^{\text{avg}}(x_i), \mu)$$

A record is considered anomalous if $\mathcal{A}(x_i)$ exceeds a predefined threshold. In our experiments, we adopt a rank-based approach and flag the top 5% of highest-scoring records as anomalies. This threshold is selected based on the average proportion of anomalies in anomaly detection datasets available in the ODDS repository [37].

Alternative Strategies. This scoring method is compatible with other strategies such as:

- **Intra-record disagreement:** using $\text{sim}(h^{(k)}, h^{(l)})$ across partitions.
- **Contextual deviation:** comparing $h^{\text{avg}}(x_i)$ to neighbors in $\mathcal{G}(x_i)$.

These metrics are unified in that they rely entirely on learned representations, preserving the self-supervised nature of the detector.

5 Implementation Pipeline for Anomaly Explanation

While anomaly detection is crucial for ensuring data quality, understanding *why* a sample is anomalous is equally important for actionable insights, model trust, and downstream correction. This is especially critical for detecting Type 2 (cross-feature inconsistencies) and Type 3 (context violations) anomalies, which do not manifest as simple outliers. We introduce a unified explanation module that leverages the same semantic embeddings used in our contrastive anomaly detection framework to identify violated dependencies, conflicting feature groups, and deviations from contextual norms—without requiring labeled data or post-hoc surrogate models.

5.1 Explanation Method Overview

Given a flagged record x and a trained model f , the explanation module proceeds in two stages:

(1) Intra-record inconsistency. We first compute the semantic embeddings $\{h^{(k)}\}_{k=1}^K$ for each partition $x^{(k)}$ of the input. From these, we construct a disagreement matrix:

$$D_{kl} = 1 - \text{cosine}(h^{(k)}, h^{(l)}), \quad \forall k, l \in \{1, \dots, K\}$$

We identify the most conflicting pair:

$$(k^*, l^*) = \arg \max_{k < l} D_{kl}$$

This highlights the pair of feature groups with the strongest semantic disagreement, indicating a likely location of the anomaly within the record.

(2) Contextual deviation. If context groups $\mathcal{G}(x)$ are defined (e.g., via k-NN or time windows), we compare the average embedding of x to the centroid of its neighbors:

$$h^{(\text{avg})}(x) = \frac{1}{K} \sum_{k=1}^K h^{(k)}, \quad \text{sim}_{\text{ctx}} = \frac{1}{|\mathcal{G}(x)|} \sum_{x' \in \mathcal{G}(x)} \text{cosine}(h^{(\text{avg})}(x), h^{(\text{avg})}(x'))$$

If sim_{ctx} falls below a threshold (e.g., 0.9), we flag the sample as contextually inconsistent.

(3) Dependency violations. Using inlier data, we compute a dependency matrix:

$$P_{kl} = \Pr[D_{kl} < \eta]$$

This quantifies how often each pair of partitions agrees in normal data. If a test-time pair (k, l) has $D_{kl}(x) > \eta$ but $P_{kl} > \rho$, we report:

“Violated: Partition k is normally consistent with Partition l .”

This provides rule-level, interpretable explanations derived directly from the training distribution.

5.2 Mutation-Based Evaluation of Explanations

To evaluate the effectiveness of our explanation module, we synthetically inject known anomaly types into held-out test samples and verify whether the explanation system recovers the correct cause.

Type-1 anomalies: A single feature is corrupted (e.g., `age = -999`). Expected explanation: self-conflict in the partition containing the corrupted feature.

Type-2 anomalies: Logically incompatible feature combinations are injected (e.g., `age = 12` with `has_license = true`). Expected explanation: conflict between two partitions that contain the pair.

Type-3 anomalies: Contextual anomalies are created by placing an out-of-place record in a neighborhood it does not belong to. Expected explanation: low context similarity.

Type-4 perturbations: Benign noise is added to random samples to test robustness. Expected: no explanation should be triggered.

5.3 Implementation Pipeline

1. **Mutation Injection.** We mutate a copy of the test set to include synthetic anomalies of types A1–A4 using a mutation rate (e.g., 5%).
2. **Expected Annotations.** For each mutation type, we define the expected explanation (e.g., which partitions should disagree).
3. **Explanation Computation.** For each mutated record, we run the `explain_record()` module to compute:
 - Most conflicting partition pair (k^*, l^*)
 - Violated dependencies from P_{kl}
 - Context alignment score (if applicable)
4. **Evaluation.** We compute explanation accuracy as the fraction of mutated records for which the expected explanation was successfully retrieved:

$$\text{Accuracy} = \frac{1}{|\mathcal{M}|} \sum_{x \in \mathcal{M}} \mathbb{1}\{(k^*, l^*) \in \text{Expected}(x)\}$$

Mutation Function for Synthetic Anomalies

To support controlled evaluation of explanation accuracy, we inject labeled anomalies into a clean test set using a mutation function tailored to simulate the four canonical anomaly types (A1–A4). The function `mutate_test_set()`

receives a mutation type (e.g., “A2”), mutation rate, and random seed, and returns a mutated version of the test set along with the indices of altered records.

For each mutation type, the procedure is as follows:

- **A1 (Type-1 Anomalies):** A single feature is randomly selected for each mutated sample and replaced with a clearly invalid value (e.g., -999). This simulates domain-violating input such as negative ages or unbounded sensor glitches.
- **A2 (Type-2 Anomalies):** Two features are randomly selected and each is replaced with a value at the opposite end of its empirical range. Specifically, if the original value is below the median of the feature distribution, it is replaced with the feature’s maximum value; otherwise, it is replaced with the minimum. This simulates logically incompatible combinations of attributes.
- **A3 (Type-3 Anomalies):** A contextual mismatch is injected by copying a distant record into a sample’s position, making it locally valid but out-of-distribution in context. This breaks contextual consistency (e.g., inserting an elderly patient into a neonatal cohort).
- **A4 (Type-4 Noise):** Harmless stochastic noise is added to the sample (drawn from $\mathcal{N}(0, 0.01)$). These perturbations should not trigger anomalies if the detector is robust.

Each mutation type is applied independently with a configurable mutation rate (e.g., 5%), and all mutations are merged into a single altered test set for explanation benchmarking. Ground truth expectations for each type (e.g., most conflicting partitions or expected context similarity drops) are used to measure how faithfully the explanation module recovers the intended anomaly structure.

6 Optimization and Convergence

Our context-aware contrastive learning framework is optimized using a principled and empirically validated approach that ensures stable convergence. This section outlines the mathematical foundations and practical strategies that enable our model to converge reliably during training.

6.1 Smooth and Differentiable Objective

We minimize a contrastive InfoNCE-style objective that promotes both intra-record and contextual consistency:

$$\mathcal{L}_{\text{ctx+mv}} = -\frac{1}{|\mathcal{B}|} \sum_{(i,k) \in \mathcal{B}} \log \frac{\sum_{(j,l) \in \mathcal{P}_{i,k}} w_{(j,l)} \exp(\text{sim}(h_i^{(k)}, h_j^{(l)})/\tau)}{\sum_{(j,l) \in \mathcal{N}_{i,k}} \exp(\text{sim}(h_i^{(k)}, h_j^{(l)})/\tau)} \quad (1)$$

This loss is fully differentiable and structured to reward alignment between semantically related embeddings (within-record and contextual neighbors) while pushing apart unrelated examples. It provides smooth gradients conducive to mini-batch optimization.

6.2 Stable Optimization with Adam

We use the Adam optimizer with default settings and a fixed learning rate ($\alpha = 0.001$), which offers per-parameter adaptive updates and is robust to gradient noise in non-convex spaces. Mini-batch training further stabilizes the updates and facilitates faster convergence.

6.3 Stochastic Context Sampling for Scalability

To reduce computational overhead and stabilize gradients, we apply stochastic context sampling. Instead of computing full pairwise losses over the entire context group $\mathcal{G}(i)$, we sample $M \ll |\mathcal{G}(i)|$ context positives per anchor. This lowers the per-batch complexity from $\mathcal{O}(B^2K)$ to $\mathcal{O}(BMK)$, improving scalability and learning stability.

6.4 Embedding Normalization and Bounded Similarity

To ensure that cosine similarity comparisons remain bounded, we apply ℓ_2 normalization to each projected embedding:

$$h^{(k)} = \frac{g_\phi(f_\theta(x^{(k)}))}{\|g_\phi(f_\theta(x^{(k)}))\|_2} \quad (2)$$

This prevents vector norm explosion or collapse and constrains the optimization landscape, which aids in consistent convergence.

6.5 Empirical Convergence Monitoring

We empirically track the training loss per epoch using an averaged batch loss:

$$\mathcal{L}_{\text{epoch}} = \frac{1}{N_{\text{batches}}} \sum_{b=1}^{N_{\text{batches}}} \mathcal{L}_{\text{ctx+mv}}^{(b)}$$

Convergence is confirmed through smooth decline or plateauing of this curve. Additionally, we evaluate the stability of learned embeddings and downstream anomaly scores.

These design decisions provide strong theoretical and practical support for optimization stability and convergence of our self-supervised framework.

References

- [1] Tharindu Fernando, Harshala Gammulle, Simon Denman, Sridha Sridharan, and Clinton Fookes. Deep learning for medical anomaly detection – a survey. *ACM Comput. Surv.*, 54(7), July 2021. ISSN 0360-0300. doi: 10.1145/3464423. URL <https://doi.org/10.1145/3464423>.
- [2] San Bin and Yuan Zhao. A comprehensive review of fraud-oriented anomaly detection. *IEEE Access*, 10: 77712–77733, 2022.
- [3] Kyuho Kim and Sang Shim. Machine-learning approaches for predictive maintenance of industrial equipment. *Engineering Applications of Artificial Intelligence*, 109:104664, 2022.
- [4] Rayees Ahmad Dar, Muzafar Rasool, Assif Assad, et al. Breast cancer detection using deep learning: Datasets, methods, and challenges ahead. *Computers in biology and medicine*, 149:106073, 2022.
- [5] Waleed Hilal, S Andrew Gadsden, and John Yawney. Financial fraud: a review of anomaly detection techniques and recent advances. *Expert systems With applications*, 193:116429, 2022.
- [6] Andrey Kharitonov, Abdulrahman Nahhas, Matthias Pohl, and Klaus Turowski. Comparative analysis of machine learning models for anomaly detection in manufacturing. *Procedia Computer Science*, 200:1288–1297, 2022. ISSN 1877-0509. doi: <https://doi.org/10.1016/j.procs.2022.01.330>. URL <https://www.sciencedirect.com/science/article/pii/S1877050922003398>.
- [7] Vic Barnett, Toby Lewis, et al. *Outliers in statistical data*, volume 3. Wiley New York, 1994.
- [8] Bernhard Schölkopf, John C Platt, John Shawe-Taylor, Alex J Smola, and Robert C Williamson. Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7):1443–1471, 2001.
- [9] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *ICDM*, pages 413–422, 2008.
- [10] Mei-Ling Shyu, Shu-Ching Chen, Kanoksri Sarinnapakorn, and LiWu Chang. Principal component-based anomaly detection scheme. In *Foundations and novel approaches in data mining*, pages 311–329. Springer, 2005.
- [11] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*, pages 4–11, 2014.
- [12] Jinwon An and Sungzoon Cho. Variational autoencoder based anomaly detection using reconstruction probability. *Special lecture on IE*, 2(1):1–18, 2015.
- [13] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.
- [14] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

- [15] Jiaxin Yin, Yuanyuan Qiao, Zitang Zhou, Xiangchao Wang, and Jie Yang. Mcm: Masked cell modeling for anomaly detection in tabular data. In *The Twelfth International Conference on Learning Representations*, 2024.
- [16] Ruiying Lu, Jinhan Liu, Chuan Du, and Dandan Guo. Investigating mask-aware prototype learning for tabular anomaly detection. *arXiv preprint arXiv:2506.02757*, 2025.
- [17] Yizhou Li, Yijie Wang, Hongzuo Xu, Bin Li, and Xiaohui Zhou. Deep anomaly detection with partition contrastive learning for tabular data. *Data Mining and Knowledge Discovery*, 39(4):30, 2025.
- [18] Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A Zuluaga. Usad: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3395–3404, 2020.
- [19] Amjad Iqbal and Rashid Amin. Time series forecasting and anomaly detection using deep learning. *Computers & Chemical Engineering*, 182:108560, 2024. ISSN 0098-1354. doi: <https://doi.org/10.1016/j.compchemeng.2023.108560>. URL <https://www.sciencedirect.com/science/article/pii/S0098135423004301>.
- [20] Jiehui Xu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Anomaly transformer: Time series anomaly detection with association discrepancy. *arXiv preprint arXiv:2110.02642*, 2021.
- [21] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “why should i trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144, 2016.
- [22] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.
- [23] Scott M Lundberg, Gabriel G Erion, and Su-In Lee. Consistent individualized feature attribution for tree ensembles. 2018. *arXiv preprint arXiv:1802.03888*, page 127, 1802.
- [24] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *International conference on machine learning*, pages 3145–3153. PMIR, 2017.
- [25] Liat Antwarg, Ronnie Mindlin Miller, Bracha Shapira, and Lior Rokach. Explaining anomalies detected by autoencoders using shapley additive explanations. *Expert systems with applications*, 186:115736, 2021.
- [26] Shengyao Zhou, Jie He, Hui Yang, Donghua Chen, and Runtong Zhang. Big data-driven abnormal behavior detection in healthcare based on association rules. *IEEE Access*, 8:129002–129011, 2020.
- [27] Ying Zhao. Logical: Towards logical anomaly synthesis for unsupervised anomaly localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4022–4031, 2024.
- [28] Yu Zhang, Ruoyu Li, Nengwu Wu, Qing Li, Xinhan Lin, Yang Hu, Tao Li, and Yong Jiang. Dissect black box: Interpreting for rule-based explanations in unsupervised anomaly detection. *Advances in Neural Information Processing Systems*, 37:84169–84196, 2024.
- [29] Hajar Homayouni, Sudipto Ghosh, and Indrakshi Ray. Adquate: An automated data quality test approach for constraint discovery and fault detection. In *2019 IEEE 20th International Conference on Information Reuse and Integration for Data Science (IRI)*, pages 61–68. IEEE, 2019.
- [30] Hajar Homayouni, Sudipto Ghosh, Indrakshi Ray, Shlok Gondalia, Jerry Duggan, and Michael G. Kahn. An autocorrelation-based lstm-autoencoder for anomaly detection on time-series data. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 5068–5077, 2020. doi: 10.1109/BigData50022.2020.9378192.
- [31] Yiheng Zhang, Yunkang Cao, Xiaohao Xu, and Weiming Shen. LogiCode: an LLM-driven framework for logical anomaly detection. *IEEE Transactions on Automation Science and Engineering*, 2024.
- [32] Karthik S Gurumoorthy, Amit Dhurandhar, and Guillermo Cecchi. ProtoDash: Fast interpretable prototype selection. *arXiv preprint arXiv:1707.01212*, 2017.
- [33] Amirata Ghorbani, James Wexler, James Y Zou, and Been Kim. Towards automatic concept-based explanations. *Advances in neural information processing systems*, 32, 2019.

- [34] Ramaravind K. Mothilal, Amit Sharma, and Chenhao Tan. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* '20, page 607–617, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450369367. doi: 10.1145/3351095.3372850. URL <https://doi.org/10.1145/3351095.3372850>.
- [35] Daniel Nemirovsky, Nicolas Thiebaud, Ye Xu, and Abhishek Gupta. CounterGAN: Generating realistic counterfactuals with residual generative adversarial nets. *arXiv preprint arXiv:2009.05199*, 2020.
- [36] Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, et al. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In *International conference on machine learning*, pages 2668–2677. PMLR, 2018.
- [37] Shebuti Rayana. ODDS: Outlier detection datasets. <https://shebuti.com/outlier-detection-datasets-odds/>, 2016. Accessed: July 30, 2025.